

작성일: 2026-07-07

이 문서는 RoK4-Isaac-Velocity-Flat-v0 task 의 현재 reward 구조와 reward function 설정을 정리한다. 현재 reward 는 Isaac Lab G1 velocity task 구조를 RoK4 링크 /관절 이름에 맞게 이식한 flat walking 초기 baseline 이다. 최종 튜닝값이 아니라 첫 보행 실험을 위한 시작점으로 봐야 한다.

관련 파일

역할	파일
RoK4 flat task 설정	<code>/home/rcclab/rok4_lab/source/rok4_tasks/ rok4_tasks/manager_based/locomotion/ velocity/config/rok4/flat_env_cfg.py</code>
공통 locomotion reward term 기본값	<code>/home/rcclab/IsaacLab/source/ isaacclab_tasks/isaacclab_tasks/ manager_based/locomotion/velocity/ velocity_env_cfg.py</code>
locomotion 전용 reward 함수	<code>/home/rcclab/IsaacLab/source/ isaacclab_tasks/isaacclab_tasks/ manager_based/locomotion/velocity/mdp/ rewards.py</code>
Isaac Lab 공통 reward 함수	<code>/home/rcclab/IsaacLab/source/isaacclab/ isaacclab/envs/mdp/rewards.py</code>
mdp namespace 연결	<code>/home/rcclab/IsaacLab/source/ isaacclab_tasks/isaacclab_tasks/ manager_based/locomotion/velocity/mdp/ __init__.py</code>

구조 요약

RoK4FlatEnvCfg

└ 상속: LocomotionVelocityRoughEnvCfg

└ 포함: rewards = RoK4RewardsCfg()

RoK4RewardsCfg

└ 상속: RewardsCfg

각 reward term

└ RewardTermCfg(func=mdp.xxx, weight=..., params=...)

mdp.xxx

└ locomotion 전용 함수: velocity/mdp/rewards.py

└ Isaac Lab 공통 함수: isaac lab/envs/mdp/rewards.py

RoK4FlatEnvCfg -> RoK4RewardsCfg 는 상속이 아니라 포함/사용 관계이고,

RoK4RewardsCfg -> RewardsCfg 는 상속 관계이다. mdp/rewards.py 는 부모

클래스가 아니라 실제 reward 계산 함수가 들어 있는 함수 모음이다.

실제로는 다음처럼 이해하면 된다.

RoK4RewardsCfg 는 RewardsCfg 를 상속받아서 기본 reward term 들을 물려받는다.

RoK4 에 필요한 reward 는 mdp.xxx 함수를 직접 지정해 새로 정의하거나 override 한다.

RewardTermCfg/RewTerm 은 mdp.xxx 함수, weight, params 를 하나의 reward term 으로 묶는다.

Reward 계산 방식

Isaac Lab 의 RewardManager 는 각 reward term 의 raw value 를 계산한 뒤 weight 를 곱하고, 모든 term 을 합산한다.

```
total_reward =  
  
    sum(weight_i * reward_function_i(env, params_i))
```

Positive weight 는 행동을 유도하고, negative weight 는 penalty 로 작동한다.

RoK4 전용 Reward Terms

term	function	weight	params	의미
termination_penalty	mdp.is_terminated	-20.0	없음	episode 가 termina

되면 큰
penalty
를 준다.

world
frame
yaw
angular
velocity
comma
nd 를
추종하게
한다.

```

command
d_name
="base
_veloc
ity",
std=0.
5

```

```
feet_air_      mdp.fe      0.      feet:
time           et_air      75      L_Foot
              _time_      _Link,
              positi      R_Foot
              ve_bip      _Link;
```

			thresh old=0. 4	있는 step pattern 을 유도한 다.
feet_slid e	mdp.fe et_sli de	- 0. 2	feet: L_Foot _Link, R_Foot _Link	지면 접촉 중인 발이 미끄러지 는 것을 줄인다.
dof_pos_l imits	mdp.jo int_po s_limi ts	- 1. 0	.*_Ank le_Pit ch_Joi nt, .*_A nkle_R oll_Jo int	ankle joint 가 soft joint limit 을 넘는 것을 강하게 막는다.
joint_dev iation_hi p	mdp.jo int_de viatio n_ll	- 0. 1	.*_Hip _Yaw_J oint, .* _Hip_R oll_Jo int	hip yaw/roll 이 default pose

				에서
				과도하게
				벗어나지
				않게
				한다.
				torso
				yaw 가
				default
joint_dev	mdp.jo	-	Torso_	pose
iation_to	int_de	0.	Yaw_Jo	에서
rso	viatio	1	int	과도하게
	n_l1			벗어나지
				않게
				한다.

부모 RewardsCfg 에서 상속받고 RoK4 에서 수정한 Terms

term	function	current weight	RoK4 설정	의미
lin_vel_z_l2	mdp.lin_vel_z_l2	-0.2	기본 robot root	base 가 z 방향으로 튀는 움직임을

				줄인다.
				roll/
				pitch
			기본	angular
ang_vel_x	mdp.ang_vel	-	robot	velocity
y_l2	_xy_l2	0.	root	를 줄인다
		05		.
				몸이
				기울어지
				는 것을
flat_orie	mdp.flat_or	-	기본	줄이고
ntation_l	ientation_l	1.	robot	upright
2	2	0	root	자세를
				유도한다.
				action
				이 step
				마다
				급격히
action_ra	mdp.action_	-	action	변하는
te_l2	rate_l2	0.	전체	것을
		00		줄인다.
		5		
dof_acc_l	mdp.joint_a	-	.*_Hip	hip/
2	cc_l2	1.	_.*,.*_	knee
		0e	Knee_P	joint
		-7	itch_J	accelera
			oint	tion을
				줄여

							움직임을 부드럽게 한다.
						<code>.*_Hip</code>	hip/
						<code>_.*,.*_</code>	knee/
						<code>Knee_P</code>	ankle
<code>dof_torques_l2</code>	<code>mdp.joint_torques_l2</code>				<code>2.0e-6</code>	<code>itch_Joint,.*_Ankle_.*</code>	torque 사용량을 줄인다.
						<code>Base_L</code>	발이 아닌
						<code>ink,</code>	base/up
						<code>Upper_</code>	per
<code>undesired_contacts</code>	<code>mdp.undesired_contacts</code>					<code>Body_L</code>	body
						<code>ink;</code>	접촉을
						<code>thresh</code>	penalty
						<code>old=1.</code>	로
						<code>0</code>	처리한다.

Reward Function 요약

function	위치	계산 의미
<code>is_terminated</code>	Isaac Lab 공통	termination manager 의 terminated flag 를 reward term 으로 반환한다.
<code>track_lin_vel_x</code> <code>y_yaw_frame_exp</code>	locomoti on mdp	yaw frame x/y 선속도 command error 에 <code>exp(-error /</code>

<code>track_ang_vel_z_world_exp</code>	locomotion mdp	std^2)를 적용한다.
<code>feet_air_time_positive_biped</code>	locomotion mdp	world z 축 angular velocity command error 에 $\exp(-\text{error} / \text{std}^2)$ 를 적용한다.
<code>feet_slide</code>	locomotion mdp	한 발 지지 상태의 air/contact time 을 보상하며, command 크기가 작으면 0 이 된다.
<code>lin_vel_z_l2</code>	Isaac Lab 공통	접촉 중인 foot 의 horizontal linear velocity norm 을 penalty 로 계산한다.
<code>ang_vel_xy_l2</code>	Isaac Lab 공통	base body-frame z velocity squared.
<code>flat_orientation_l2</code>	Isaac Lab 공통	base body-frame roll/pitch angular velocity squared sum.
<code>joint_torques_l2</code>	Isaac Lab 공통	projected gravity 의 x/y component squared sum 으로 몸 기울기를 penalty 화한다.
<code>joint_acc_l2</code>	Isaac Lab 공통	선택된 joint 의 applied torque squared sum.
<code>action_rate_l2</code>	Isaac Lab 공통	선택된 joint 의 joint acceleration squared sum.
	Isaac Lab 공통	현재 action 과 이전 action 차이의 squared sum.

<code>joint_deviation_l1</code>	Isaac Lab 공통	현재 joint position 과 default joint position 차이의 absolute sum.
<code>joint_pos_limits</code>	Isaac Lab 공통	soft joint position limit 을 넘은 정도를 합산한다.
<code>undesired_contacts</code>	Isaac Lab 공통	지정 body 의 contact force 가 threshold 를 넘은 contact 개수를 penalty 로 반환한다.

Command 와 Reward 의 연결

```
lin_vel_x = (0.0, 0.8)

lin_vel_y = (-0.3, 0.3)

ang_vel_z = (-0.6, 0.6)
```

이 command 는 `track_lin_vel_xy_exp`, `track_ang_vel_z_exp`, `feet_air_time` 에 직접 영향을 준다. 특히 `feet_air_time_positive_biped` 는 x/y command norm 이 0.1 이하이면 reward 를 0 으로 만든다.

Termination 과 Reward 의 연결

```
self.terminations.base_contact.params["sensor_cfg"].body_names =
["Base_Link", "Upper_Body_Link"]
```

`Base_Link` 또는 `Upper_Body_Link` 가 지면과 강하게 접촉하면 episode termination 이 발생하고, `termination_penalty = -200.0` 가 함께 작동한다.

현재 설계 의도

- 1. `command velocity`를 따라간다.
- 2. `upright` 자세를 유지한다.
- 3. 한 발씩 드는 `biped stepping pattern`을 만든다.
- 4. 발 미끄러짐을 줄인다.
- 5. `torque`, `joint acceleration`, `action rate`를 줄여 움직임을 부드럽게 한다.
- 6. `ankle limit`, `hip/torso deviation`을 제한한다.
- 7. `base/upper body` 접촉을 넘어짐으로 처리한다.

튜닝 시 우선 확인할 항목

현상	먼저 볼 항목
너무 자주 넘어진다	<code>termination_penalty</code> , <code>base_contact</code> , <code>flat_orientation_l2</code> , 초기 자세
거의 걷지 않고 버틴다	<code>track_lin_vel_xy_exp</code> , <code>command range</code> , <code>feet_air_time</code>
발이 많이 미끄러진다	<code>feet_slide</code> , <code>foot collision</code> , <code>friction</code> , <code>contact sensor</code>
관절이 떨린다	<code>action_rate_l2</code> , <code>dof_acc_l2</code> , PD gain, <code>action scale</code>

토크가

과도하다

dof_torques_l2, effort limit, action scale

발목이 limit

근처로 간다

dof_pos_limits, ankle default pose, action scale